RECETARIO WEB

Proyecto de Desarrollo Web — Entrega 1

INTEGRANTES

Angely Liced Herrera Roldán

Juan Camilo Barragán Pottes

GRUPO 500

04/11/2026

1. PROPÓSITO 3
1.1. ALCANCE 3
1.2. DEFINICIONES 3
1.3. REFERENCIAS 4
2. PROBLEMA A RESOLVER 4
3. JUSTIFICACIÓN 5
4. OBJETIVOS 5
4.1. OBJETIVO GENERAL 5
4.2. OBJETIVO ESPECÍFICO 6
4.3. PLAN DE TRABAJO PARA LOS SIGUIENTES ENTREGABLES 6
5. ALCANCE DEL PROYECTO 7
5.1. Funcionalidades incluidas en la Entrega 1 7
5.2. Funcionalidades previstas para la Entrega 2 8
5.3. Funcionalidades previstas para la Entrega 3 y definitiva 8
5.4. Funcionalidades fuera del alcance 9
5.5. Restricciones de la primera entrega 9
5.6. Supuestos 9
6. DESCRIPCIÓN GENERAL DEL SISTEMA 10
6.1. Perspectiva del producto 10
6.2. Características principales 10
6.3. Tipos de usuarios 10
6.4. Entorno de funcionamiento 11
7. REQUISITOS ESPECÍFICOS 11
7.1. Requisitos funcionales 11
7.2. Requisitos no funcionales 12
7.3. Modelo de datos actual 13
8. TECNOLOGÍAS UTILIZADAS 13
9. ARQUITECTURA DEL SISTEMA 14
10. DIAGRAMAS UML 15
10.1. Diagramas De Casos De uso 15
10.2. Diagrama De Componentes 16
10.3. Diagrama De Despliegue Actual 17
11. BACKLOG 18
11.1. EPICAS 18
11.2. HISTORIAS DE USUARIO 19
11.3. Must have Funcionalidades de la Entrega 1 19

.

1. PROPÓSITO

Este documento describe los requisitos del sistema Recetario, una aplicación web desarrollada para facilitar la organización y administración de recetas de cocina. La plataforma permite registrar nuevas recetas, consultar su información, realizar modificaciones y eliminarlas cuando sea necesario.

Además, cuenta con un módulo de votación que permite a los usuarios elegir entre diferentes opciones relacionadas con cada receta y consultar los resultados obtenidos.

1.1. ALCANCE

Este documento presenta la propuesta del sistema Recetario para la Entrega 1 del proyecto académico. Incluye la descripción del problema, la justificación, los objetivos, los requisitos funcionales y no funcionales, las tecnologías utilizadas y una visión general de su arquitectura.

También se describen las funcionalidades que ya se encuentran disponibles y aquellas que se tienen previstas para las siguientes entregas.

Es importante saber que las funcionalidades correspondientes a las Entregas 2, 3 y definitiva están pensadas para la evolución del proyecto, por lo que no se ve su implementación en la primera fase.

1.2. DEFINICIONES

Término	Definición

Receta	Información de una preparación de cocina que incluye nombre, categoría, ingredientes y pasos de elaboración.
Opción	Alternativa asociada a una receta sobre la cual los usuarios pueden votar.
CRUD	Operaciones básicas para crear, consultar, actualizar y eliminar información.
SRS	Operaciones básicas para crear, consultar, actualizar y eliminar información.
Django Admin	Herramienta administrativa de Django que permite gestionar los datos de la aplicación.

1.3. REFERENCIAS

1.3.1. Documentación oficial de Django 5.1.

1.3.2. IEEE Std 830-1998, utilizado como referencia para la organización del documento de requisitos.

1.3.3. Código fuente del proyecto, especialmente las carpetas **recetas/** y **recetario/**

2. PROBLEMA A RESOLVER

Actualmente, muchas personas guardan sus recetas de cocina en diferentes lugares, como cuadernos, conversaciones de WhatsApp, documentos o notas personales. Aunque estas alternativas pueden ser útiles, con el tiempo resulta complicado mantener toda la información organizada.

2.1. Esta situación puede ocasionar problemas como:

2.2. Pérdida de recetas o información importante.

2.3. Repetición de registros y dificultad para encontrar una preparación específica.

2.4. Falta de organización al momento de clasificar las recetas.

2.5. Dificultad para comparar diferentes alternativas o preferencias relacionadas con una preparación.
2.6. Ausencia de un espacio donde la información pueda consultarse y actualizarse fácilmente.

A partir de esta necesidad surge Recetario, una aplicación web que busca reunir las recetas en un solo lugar, facilitando su administración y consulta. Adicionalmente, incorpora un sistema de votación para que los usuarios puedan expresar sus preferencias entre las opciones disponibles.

3. JUSTIFICACIÓN

El desarrollo de Recetario busca ofrecer una alternativa sencilla para organizar y administrar recetas de cocina mediante una plataforma web.

La aplicación permite mantener la información en un solo lado, evitando que las recetas se encuentren dispersas en diferentes archivos o conversaciones. También facilita su consulta y actualización sin necesidad de instalar programas adicionales, ya que se puede acceder desde un navegador.

Otro aspecto importante es el módulo de votación, que permite conocer las preferencias de los usuarios frente a las opciones disponibles para cada receta.

Desde el lado del nivel académico, este proyecto representa una oportunidad para aplicar los conocimientos adquiridos durante la formación en desarrollo de software, especialmente en el manejo de bases de datos, programación backend, desarrollo web y documentación de proyectos.

Se eligió Django porque ofrece herramientas que facilitan el desarrollo de aplicaciones web, como el manejo de modelos, formularios, plantillas y un panel administrativo integrado.

4. OBJETIVOS

4.1. OBJETIVO GENERAL

Desarrollar la propuesta inicial del sistema Recetario, estableciendo sus requisitos, estructura y diseño general, y dejando como base el

funcionamiento del módulo de gestión de recetas y votaciones que ya se encuentra desarrollado.

Para esta primera entrega se busca documentar el proyecto, organizar sus funcionalidades y presentar una versión inicial que permita evidenciar el funcionamiento de sus principales características.

4.2. OBJETIVO ESPECÍFICO

4.2.1. Identificar y documentar los requisitos funcionales y no funcionales del sistema.

4.2.2. Representar la estructura y el funcionamiento de la aplicación mediante diagramas UML.

4.2.3. Organizar las funcionalidades del proyecto en un backlog, teniendo en cuenta su prioridad y las entregas previstas.

4.2.4. Presentar el funcionamiento actual del registro, consulta, edición y eliminación de recetas.

4.2.5. Comprobar el funcionamiento del módulo de votaciones y la consulta de sus resultados.

4.2.6. Publicar el código fuente en un repositorio de GitHub para facilitar su consulta y seguimiento.

4.2.7. Preparar un video corto en el que se explique el propósito del proyecto y se presenten sus principales características.

4.3. PLAN DE TRABAJO PARA LOS SIGUIENTES ENTREGABLES

Fase	Objetivo	Actividades previstas
Entrega 1	Definición y presentación inicial del proyecto.	Documento SRS, diagramas UML, backlog, repositorio GitHub y video de presentación.
Entrega 2	Fortalecer las funcionalidades principales del sistema.	Registro e inicio de sesión, manejo de roles, ajustes en Django Admin, diagramas de secuencia, Docker, primer despliegue y actualización de la documentación.
Entrega 3	Ampliar la arquitectura e incorporar nuevas	Desarrollo de un microservicio con Node.js, documentación mediante Swagger/OpenAPI,

	funcionalidades.	integración con Django, incorporación de IA generativa y Docker Compose.
Entrega definitiva	Presentar la versión final del proyecto.	Ajustes generales, documentación final, enlaces de los servicios desplegados y video de sustentación.

5. ALCANCE DEL PROYECTO

5.1. Funcionalidades incluidas en la Entrega 1

ID	Funcionalidad
F-01	Consultar el listado de recetas registradas.
F-02	Ver la información completa de una receta, incluyendo ingredientes, preparación y opciones disponibles.
F-03	Registrar nuevas recetas mediante un formulario.
F-04	Modificar la información de una receta existente.
F-05	Eliminar recetas después de confirmar la acción.
F-06	Seleccionar y votar por una opción asociada a una receta.
F-07	Consultar los resultados de las votaciones.
F-08	Gestionar las recetas y sus opciones desde el panel administrativo de Django.
F-09	Elaborar y presentar la documentación del proyecto, los diagramas UML, el backlog, el repositorio y el video.

5.2. Funcionalidades previstas para la Entrega 2

En la segunda entrega se tiene previsto mejorar el funcionamiento general de la aplicación e incorporar herramientas que permitan controlar mejor el acceso y preparar el sistema para su despliegue.

Entre las actividades planteadas se encuentran:

5.2.1. Implementar el registro y el inicio de sesión de usuarios.
5.2.2. Incorporar roles y permisos para diferenciar los tipos de usuario.
5.2.3. Mejorar la administración del sistema.
5.2.4. Elaborar los diagramas de secuencia.
5.2.5. Crear el Dockerfile para ejecutar la aplicación en un contenedor.
5.2.6. Realizar el primer despliegue en un entorno de nube.
5.2.7. Actualizar el documento SRS e incluir una matriz de trazabilidad.
5.2.8. Preparar un video demostrativo de las nuevas funcionalidades.

5.3. Funcionalidades previstas para la Entrega 3 y definitiva

Para las siguientes etapas se busca ampliar las capacidades del proyecto mediante la incorporación de nuevos servicios y herramientas.

Se contemplan las siguientes actividades:

5.3.1. Desarrollar un microservicio independiente utilizando Node.js
5.3.2. Documentar los servicios mediante Swagger/OpenAPI.
5.3.3. Integrar el microservicio con la aplicación desarrollada en Django.
5.3.4. Incorporar funcionalidades basadas en inteligencia artificial generativa.
5.3.5. Configurar Docker Compose para ejecutar los diferentes servicios.
5.3.6. Actualizar la documentación técnica y funcional.

5.3.7. Realizar los ajustes necesarios para presentar la versión definitiva del sistema.

5.4. Funcionalidades fuera del alcance

Por el momento, el proyecto no contempla el desarrollo de una aplicación móvil nativa ni la implementación de sistemas de notificaciones push o infraestructura especializada para almacenar y distribuir imágenes a gran escala.

5.5. Restricciones de la primera entrega

Para mantener el desarrollo enfocado en los objetivos de esta fase, se establecen las siguientes condiciones:

5.5.1. El lenguaje principal será Python.
5.5.2. Se utilizará Django 5.1 como framework de desarrollo.
5.5.3. SQLite será el motor de base de datos durante el desarrollo local.
5.5.4. La interfaz se construirá utilizando las plantillas de Django.
5.5.5. No se implementarán contenedores Docker ni microservicios en esta etapa.
5.5.6. El desarrollo estará orientado a cumplir los requisitos establecidos para la Entrega 1.

5.6. Supuestos

Para el desarrollo inicial se tienen en cuenta las siguientes condiciones:

5.6.1. Cada integrante podrá trabajar desde su propio entorno de desarrollo local.
5.6.2. Las opciones relacionadas con las recetas podrán ser creadas y administradas desde Django Admin.
5.6.3. Para esta primera versión será suficiente contar con una interfaz sencilla que permita utilizar las funcionalidades principales.
5.6.4. La estructura del proyecto se organizará de manera que permita incorporar posteriormente autenticación, Docker y servicios independientes sin tener que reconstruir toda la aplicación.

6. DESCRIPCIÓN GENERAL DEL SISTEMA

6.1. Perspectiva del producto

Recetario es una aplicación web desarrollada con Django. Su estructura está organizada en dos partes principales: el proyecto recetario, encargado de la configuración general, y la aplicación recetas, donde se encuentran los modelos, las vistas, las rutas y las plantillas relacionadas con el funcionamiento del sistema.

En esta primera versión se utiliza una arquitectura monolítica, lo que significa que las funcionalidades principales se encuentran dentro de una misma aplicación.

6.2. Características principales

El sistema cuenta con las siguientes características:

6.2.1. **Administración de recetas:** permite registrar, consultar, modificar y eliminar recetas.

6.2.2. **Gestión de opciones:** permite asociar diferentes alternativas a una receta.

6.2.3. **Sistema de votación:** permite seleccionar una opción y registrar el voto correspondiente.

6.2.4. **Consulta de resultados:** muestra la cantidad de votos obtenidos por cada opción.

6.2.5. **Panel administrativo:** permite gestionar la información almacenada mediante Django Admin.

6.3. Tipos de usuarios

Actor	Descripción
Visitante	Puede consultar las recetas, crear nuevas, modificar o eliminar registros y participar en las votaciones disponibles. No necesita iniciar sesión para utilizar estas funcionalidades en la versión actual.
Administrador	Accede al panel /admin/ de Django para gestionar las recetas, las opciones y los demás registros disponibles en el sistema.

6.4. Entorno de funcionamiento

La aplicación está diseñada para ejecutarse en los sistemas operativos Windows, Linux y macOS, siempre que cuenten con Python y las dependencias necesarias.

Para utilizarla se requiere:

6.4.1. Un navegador web actualizado, como Chrome, Edge o Firefox.
6.4.2. Python instalado en el equipo.
6.4.3. Django 5.1.
6.4.4. Una base de datos SQLite.
6.4.5. Ejecutar el servidor local mediante el comando python manage.py runserver.

La información se almacena en el archivo db.sqlite3 durante esta etapa de desarrollo.

7. REQUISITOS ESPECÍFICOS

7.1. Requisitos funcionales

ID	Descripción	Prioridad
RF-01	El sistema debe mostrar un listado de las recetas registradas y permitir acceder a su información.	Alta
RF-02	El sistema debe mostrar los datos de cada receta, incluyendo nombre, categoría, ingredientes y preparación.	Alta
RF-03	El sistema debe permitir registrar nuevas recetas mediante un formulario.	Alta
RF-04	El sistema debe permitir modificar la información de las recetas existentes.	Alta
RF-05	El sistema debe permitir eliminar una receta, solicitando confirmación	Alta

	antes de completar la acción.	
RF-06	El sistema debe permitir seleccionar una opción asociada a una receta y registrar el voto correspondiente.	Media
RF-07	El sistema debe mostrar la cantidad de votos obtenidos por cada opción.	Media
RF-08	El sistema debe validar que el usuario haya seleccionado una opción antes de registrar un voto.	Media
RF-09	El administrador debe poder gestionar los registros de recetas y opciones desde Django Admin.	Baja

7.2. Requisitos no funcionales

ID	Descripción	Categoría
RNF-01	Las operaciones habituales de consulta y administración de recetas deben responder en menos de dos segundos en el entorno local de desarrollo.	Rendimiento
RNF-02	Los formularios que utilicen solicitudes POST deben contar con protección CSRF proporcionada por Django.	Seguridad
RNF-03	El código fuente debe mantenerse organizado y versionado mediante Git y GitHub.	Mantenibilidad
RNF-04	La interfaz debe permitir utilizar las funcionalidades principales desde un navegador de escritorio. Se buscará mejorar su adaptación a otros dispositivos en futuras versiones.	Usabilidad
RNF-05	La documentación del proyecto, incluyendo el SRS, los diagramas UML y el backlog, debe mantenerse organizada dentro de la	Documentación

	carpeta docs/.	

7.3. Modelo de datos actual

La primera versión del sistema utiliza dos modelos principales para almacenar la información.

Modelo Receta

Contiene los datos básicos de cada preparación:

- id: identificador único de la receta.
- nombre: nombre de la preparación.
- categoría: categoría a la que pertenece.
- ingredientes: descripción de los ingredientes necesarios.
- preparación: instrucciones para elaborar la receta.

Modelo Opción

Almacena las alternativas de votación relacionadas con cada receta:

- id: identificador único de la opción.
- Receta: relación con el registro correspondiente del modelo Receta mediante una llave foránea.
- opcion_texto: descripción de la opción disponible.
- votos: cantidad de votos registrados.

8. TECNOLOGÍAS UTILIZADAS

Componente	Tecnología	Uso dentro del proyecto
Lenguaje de programación	Python 3.x	Desarrollo de la lógica de la aplicación.
Framework	Django 5.1	Estructura y funcionamiento de la aplicación web.
Base de datos	SQLite	Almacenamiento de recetas y opciones durante el desarrollo

		local.
Interfaz	HTML y Django Templates	Construcción de las páginas y formularios.
Administración	Django Admin	Gestión de los registros del sistema.
Control de versiones	Git y GitHub	Seguimiento de cambios y almacenamiento del código fuente.
Diagramas	Mermaid	Elaboración de los diagra

9. ARQUITECTURA DEL SISTEMA

Para esta primera entrega se utiliza una arquitectura monolítica basada en el patrón de desarrollo de Django, conocido como Model–View–Template (MVT).

La aplicación se encuentra organizada de la siguiente manera:

9.1. **Proyecto recetario:** contiene la configuración general, las rutas principales y los archivos necesarios para ejecutar la aplicación.

9.2. **Aplicación recetas:** concentra la lógica relacionada con las recetas, incluyendo modelos, vistas, formularios, rutas y plantillas.

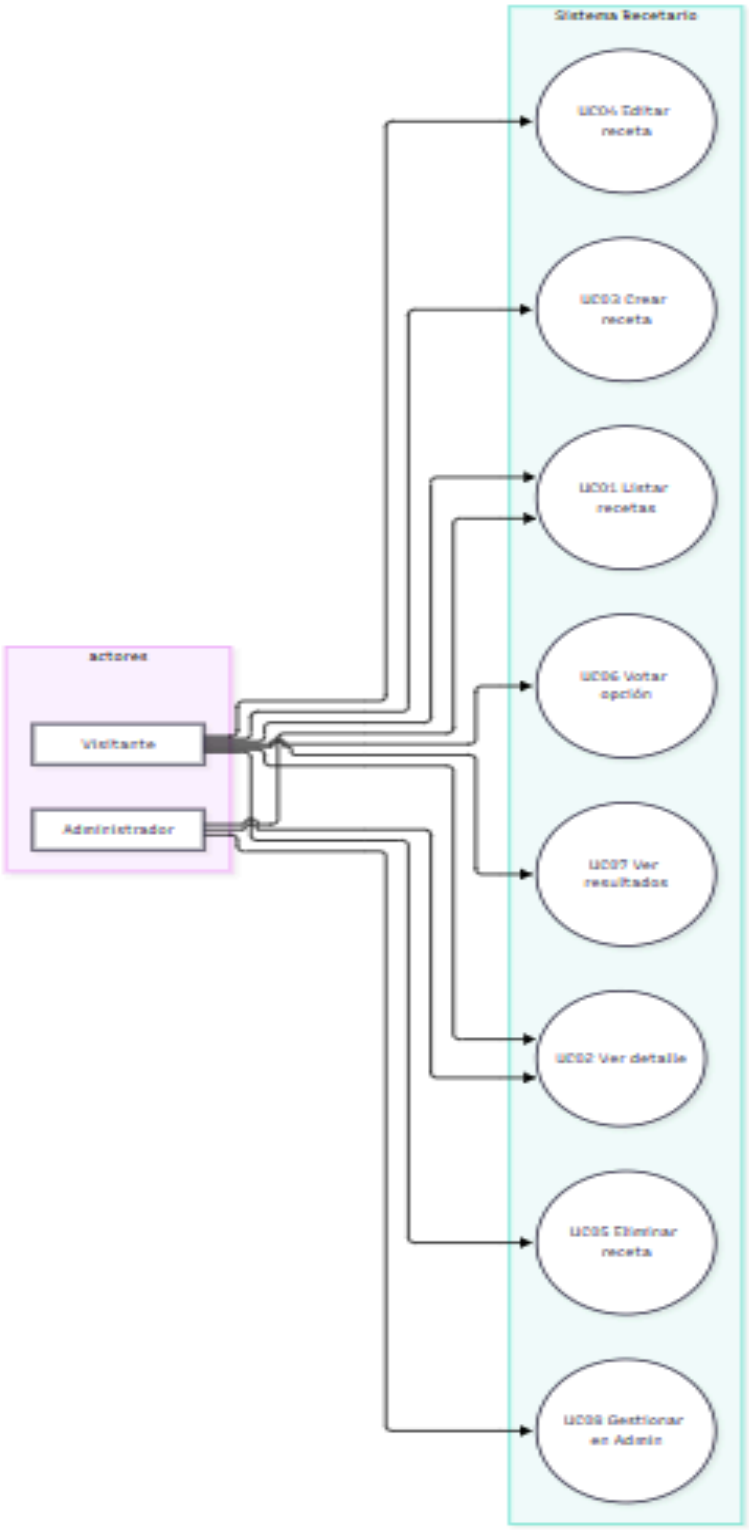
9.3. **Base de datos SQLite:** almacena la información de las recetas y sus respectivas opciones.

Durante esta etapa, el sistema se ejecuta localmente mediante el servidor de desarrollo de Django.

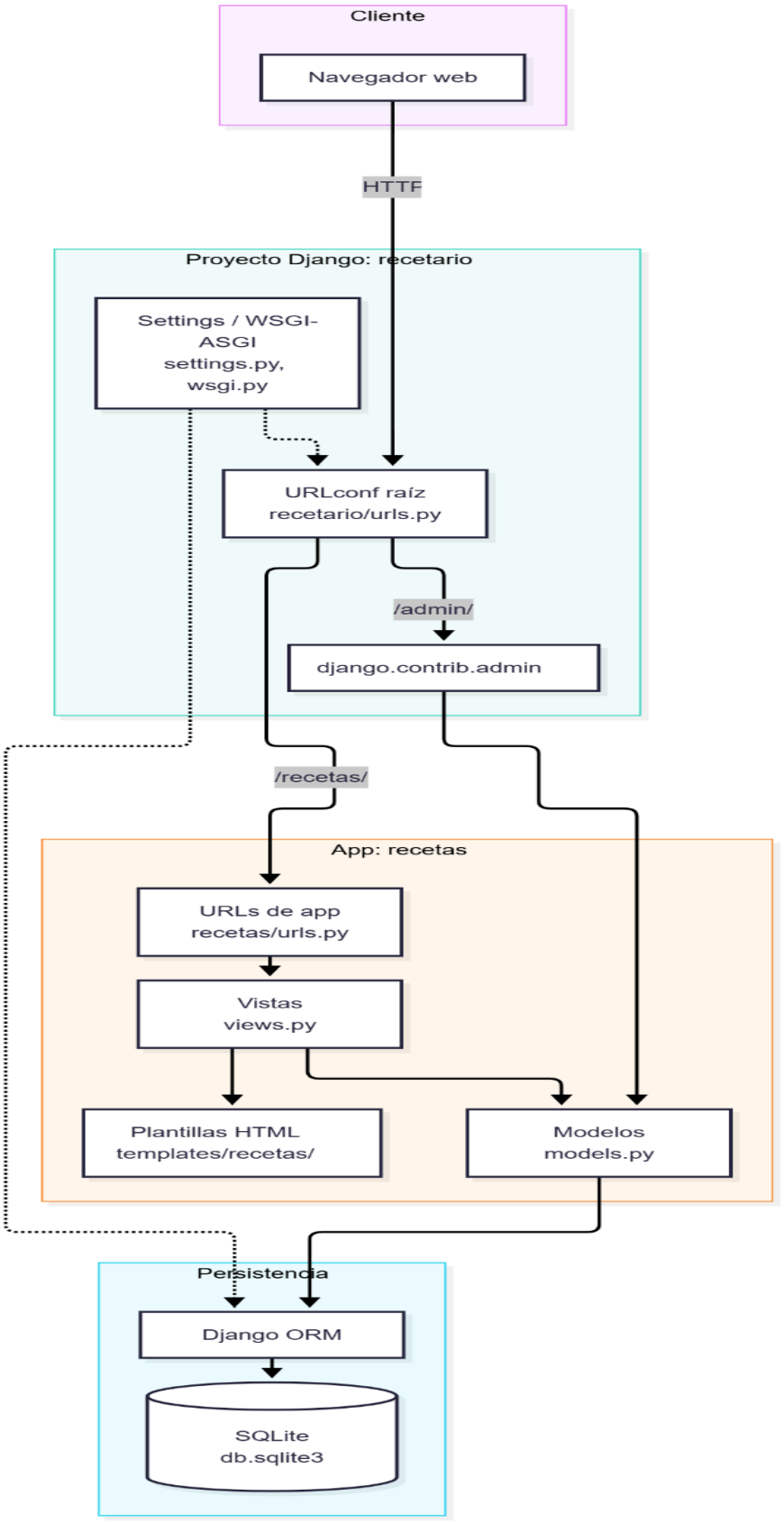
La arquitectura se ha planteado pensando en las siguientes entregas. De esta manera, será posible incorporar autenticación, contenedores y un microservicio independiente sin afectar el funcionamiento de las funcionalidades actuales.

10. DIAGRAMAS UML

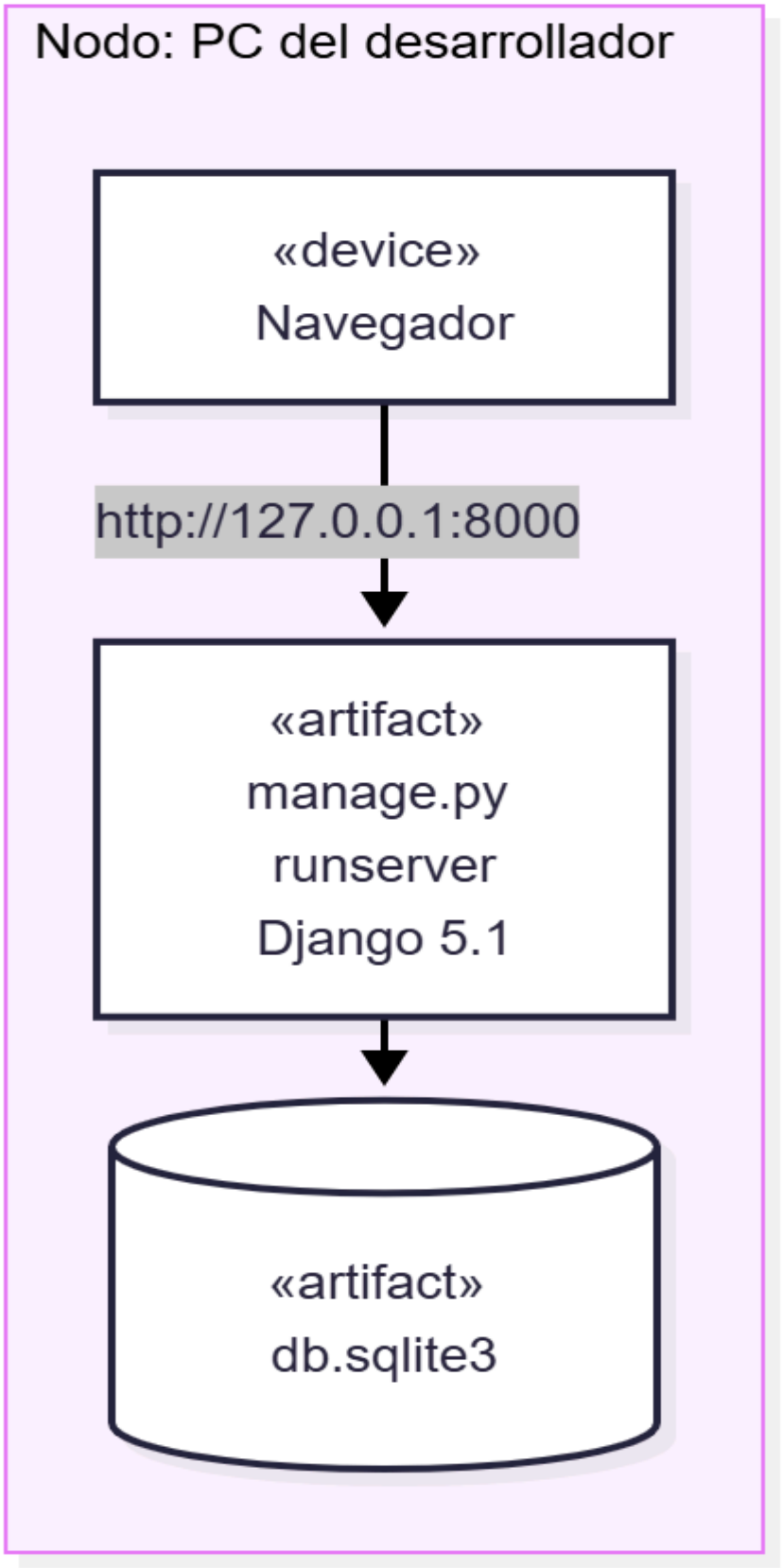
10.1. Diagramas De Casos De uso



10.2. Diagrama De Componentes



10.3. Diagrama De Despliegue Actual



11. BACKLOG

11.1. EPICAS

ID	Épica	Objetivo	Fase
E1	Gestión de recetas	Permitir registrar, consultar, actualizar y eliminar recetas.	Entrega 1
E2	Votación	Permitir votar por las opciones disponibles y consultar los resultados.	Entrega 1
E3	Administración	Gestionar recetas y opciones desde el panel administrativo de Django.	Entrega 1
E4	Calidad y entrega	Preparar la documentación, los diagramas, el repositorio y la presentación del proyecto.	Entrega 1
E5	Autenticación y roles	Incorporar registro, inicio de sesión y permisos según el tipo de usuario.	Entrega 2
E6	Contenedores y despliegue	Preparar la aplicación para ejecutarse mediante contenedores y publicarse en un entorno de despliegue.	Entrega 2
E7	Microservicios e inteligencia artificial	Incorporar un servicio desarrollado en Node.js, documentación de API e integración con inteligencia artificial.	Entrega 3

11.2. HISTORIAS DE USUARIO

Las historias de usuario describen las necesidades del sistema desde el punto de vista de quienes lo utilizan. Cada una cuenta con criterios de aceptación que permiten comprobar si la funcionalidad cumple con lo esperado.

La prioridad se organiza mediante MoSCoW:

11.2.1. **Must have:** funcionalidades necesarias para la entrega.

11.2.2. **Should have:** actividades importantes que deben completarse cuando sea posible.

11.2.3. **Could have:** mejoras opcionales.

11.2.4. **Won't have:** funcionalidades que no se desarrollarán dentro del alcance definido para el curso.

11.3. Must have Funcionalidades de la Entrega 1

ID	Historia de usuario	Épica	Criterios de aceptación	Estimación	Estado
HU-01	Como visitante, quiero consultar el listado de recetas para elegir cuál quiero revisar.	E1	Al ingresar a /recetas/, se muestra el listado de recetas con su nombre, categoría y enlace al detalle.	2	Hecho
HU-02	Como visitante, quiero consultar el detalle de una receta para conocer sus ingredientes y preparación.	E1	Al ingresar a una receta existente, se muestran sus ingredientes y preparación. Si el identificador no existe, se presenta una respuesta 404.	2	Hecho
HU-03	Como	E1	El formulario	3	Hecho

	visitante, quiero registrar una receta para guardar su información en el sistema.		permite ingresar nombre, categoría, ingredientes y preparación. Al guardar, la receta queda almacenada y el usuario regresa al listado.		
HU-04	Como visitante, quiero editar una receta para corregir o actualizar su información.	E1	El formulario muestra los datos actuales de la receta y permite modificarlos. Al guardar, los cambios quedan registrados.	3	Hecho
HU-05	Como visitante, quiero eliminar una receta con una confirmación previa para evitar borrarla por accidente.	E1	Antes de eliminar, se muestra una pantalla de confirmación. La eliminación se realiza mediante POST y, al finalizar, se regresa al listado.	2	Hecho
HU-06	Como visitante, quiero votar por una opción de una receta para expresar mi preferencia.	E2	El usuario debe seleccionar una opción. Si no selecciona ninguna, se muestra un mensaje de error. Si selecciona una, el voto se registra y se actualiza el	3	Hecho

			resultado.		
HU-07	Como visitante, quiero consultar los resultados de las votaciones para comparar las opciones disponibles.	E2	En /recetas/{id}/resultados/ se muestran las opciones de la receta junto con el total de votos de cada una.	2	Hecho
HU-08	Como administrador, quiero gestionar recetas y opciones desde el panel de Django para mantener organizada la información del sistema.	E3	Los modelos están registrados en Django Admin y permiten realizar operaciones de creación, consulta, actualización y eliminación desde /admin/.	2	Hecho
HU-16	Como equipo, queremos contar con el documento SRS, los diagramas UML y el backlog para presentar de forma clara la solución propuesta.	E4	La documentación se encuentra organizada y corresponde con las características actuales del proyecto.	Hecho	HU-16
HU-17	Como equipo, queremos contar con un repositorio en GitHub para almacenar y mantener actualizado el código y la documentación.	E4	El repositorio está creado y contiene el código fuente y los documentos del proyecto.	Hecho	HU-17